

Universidad de Valladolid

Diseño de un prototipo de consulta paralela sobre datos enlazados comprimidos

Ingeniería informática

Tecnologías de la información y la comunicación

Nombre Apellido

Tutor: Nombre Apellido

27 de octubre de 2026

Resumen

Este trabajo presenta un prototipo ficticio que reparte consultas sobre datos enlazados comprimidos entre varios procesos. Se describe su arquitectura, se implementa en Python y se evalúa con datos ilustrativos. La memoria sirve como ejemplo de documento largo para practicar la modularización de proyectos en LaTeX.

Abstract

This work presents a fictitious prototype that distributes queries over compressed linked data among several processes. It is only an example document used to practise how to split a LaTeX project into modules.

Agradecimientos

A quienes han hecho posible este taller.

Índice

Resumen I

Abstract II

Agradecimientos III

1. Introducción 1
- 1.1. Motivación . . . 1
- 1.2. Objetivos . . . 1
- 1.3. Estructura de la memoria . . . 1

2. Estado del arte 2
- 2.1. Programación paralela . . . 2
- 2.2. Datos enlazados y HDT . . . 2
- 2.3. Modularidad del software . . . 2
- 2.4. Comparación de enfoques . . . 2

3. Diseño 3
- 3.1. Requisitos . . . 3
- 3.2. Arquitectura . . . 3
- 3.3. Modelo de rendimiento . . . 3

4. Implementación 4
- 4.1. Particionado de los datos . . . 4
- 4.2. Ejecución paralela . . . 4
- 4.3. Uso desde la terminal . . . 4

5. Resultados 5

6. Conclusiones 6
- 6.1. Conclusiones . . . 6
- 6.2. Trabajo futuro . . . 6

A. Instalación 7

B. Conjuntos de datos 8

Bibliografía 9

Índice de figuras

2.1. Componentes de HDT. 2

3.1. Arquitectura del prototipo. 3

5.1. Aceleración frente al número de procesos. 5

Índice de cuadros

2.1. Comparación cualitativa (ilustrativa). 2

3.1. Requisitos del prototipo (ficticios). 3

5.1. Tiempos y métricas (datos ficticios). 5

B.1. Conjuntos de datos de prueba (ficticios). 8

1 Introducción

1.1 Motivación

La programación paralela es hoy una herramienta cotidiana, pero escribir código paralelo eficiente y portable sigue siendo difícil. Una línea de trabajo de la Universidad de Valladolid propone describir los programas con especificaciones de alto nivel y dejar que un compilador modular genere el código final [1].

> **Aviso**
>
> Esta memoria es un ejemplo para el taller: las referencias son reales; el resto, incluidos los datos, es ficticio.

1.2 Objetivos

El objetivo general es diseñar y evaluar un prototipo de consulta paralela. Los objetivos específicos son:

1. estudiar el estado del arte en paralelismo y datos enlazados;
2. diseñar una arquitectura modular;
3. implementar el prototipo en Python;
4. medir su aceleración con distinto número de procesos.

1.3 Estructura de la memoria

El capítulo 2 revisa el estado del arte; el capítulo 3 presenta el diseño; el capítulo 4 detalla la implementación; el capítulo 5 muestra los resultados y el capítulo 6 expone las conclusiones.

2 Estado del arte

2.1 Programación paralela

Trasgo es un sistema de programación paralela anidada desarrollado en la Universidad de Valladolid [1]. Su sitio web describe el grupo y sus proyectos [2].

2.2 Datos enlazados y HDT

El formato HDT divide un conjunto de datos RDF en tres componentes: cabecera, diccionario y tripletas [3].



Figura 2.1: Componentes de HDT.

2.3 Modularidad del software

Parnas defendió que un sistema debe dividirse en módulos según las decisiones de diseño que cada uno oculta [4]. Es la misma idea que aplicamos a un documento LaTeX: un fichero por responsabilidad. Las guías clásicas de LaTeX recogen estas prácticas [5].

2.4 Comparación de enfoques

Enfoque	Ventaja	Limitación
Hilos manuales	Control total	Código propenso a errores
Compilador modular	Portabilidad	Curva de aprendizaje
Datos comprimidos	Menos memoria	Coste de descompresión

Cuadro 2.1: Comparación cualitativa (ilustrativa).

3 Diseño

3.1 Requisitos

La tabla 3.1 recoge los requisitos del prototipo.

ID	Descripción	Prioridad
R1	Aceptar consultas sobre un fichero HDT	Alta
R2	Repartir el trabajo entre N procesos	Alta
R3	Informar del tiempo de ejecución	Media

Cuadro 3.1: Requisitos del prototipo (ficticios).

3.2 Arquitectura

La figura 3.1 muestra los módulos: la consulta llega a un planificador que reparte el trabajo entre procesos *worker*, que leen el almacén HDT.

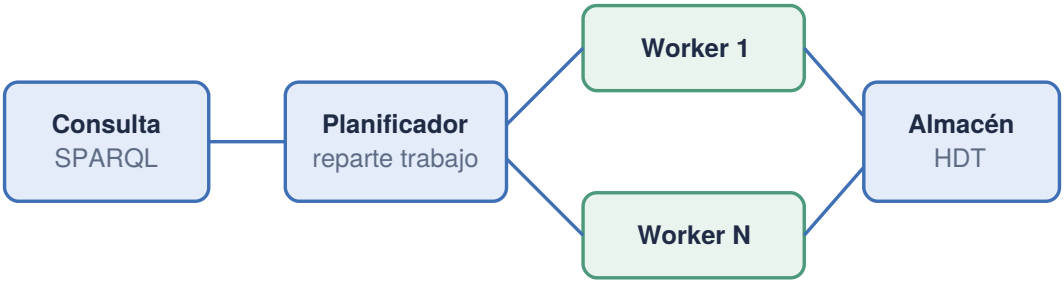


Figura 3.1: Arquitectura del prototipo.

3.3 Modelo de rendimiento

Definición 3.1 (Aceleración y eficiencia)**.** Sea $T(p)$ el tiempo con p procesos. La *aceleración* es $S(p) = T(1)/T(p)$ y la *eficiencia* es $E(p) = S(p)/p$.

Si una fracción s del trabajo es secuencial, el tiempo se modela como

$$T(p) = s\,T(1) + (1 - s)\,\frac{T(1)}{p}. \tag{3.1}$$

Ejemplo 3.1. Con $s = 0{,}1$ y $p = 8$, la ecuación (3.1) da $T(8) = 0{,}1\,T(1) + 0{,}9\,T(1)/8 \approx 0{,}21\,T(1)$.

4 Implementación

4.1 Particionado de los datos

```
def particionar(tripletas, n_workers):
    """Reparte las tripletas entre n_workers por hash del sujeto."""
    bloques = [[] for _ in range(n_workers)]
    for s, p, o in tripletas:
        bloques[hash(s) % n_workers].append((s, p, o))
    return bloques
```

4.2 Ejecución paralela

```
from concurrent.futures import ProcessPoolExecutor

def consulta_paralela(bloques, filtro):
    with ProcessPoolExecutor() as pool:
        parciales = pool.map(filtro, bloques)
    return [t for parcial in parciales for t in parcial]
```

4.3 Uso desde la terminal

```
$ python consulta.py --workers 8 --datos datos.hdt
Resultados: 1024 tripletas en 20.0 s
```

5 Resultados

La tabla 5.1 y la figura 5.1 muestran los resultados (datos ficticios) con 1, 2, 4 y 8 procesos.

Procesos	Tiempo (s)	Aceleración	Eficiencia
1	120	1,00	1,00
2	63	1,90	0,95
4	34	3,53	0,88
8	20	6,00	0,75

Cuadro 5.1: Tiempos y métricas (datos ficticios).

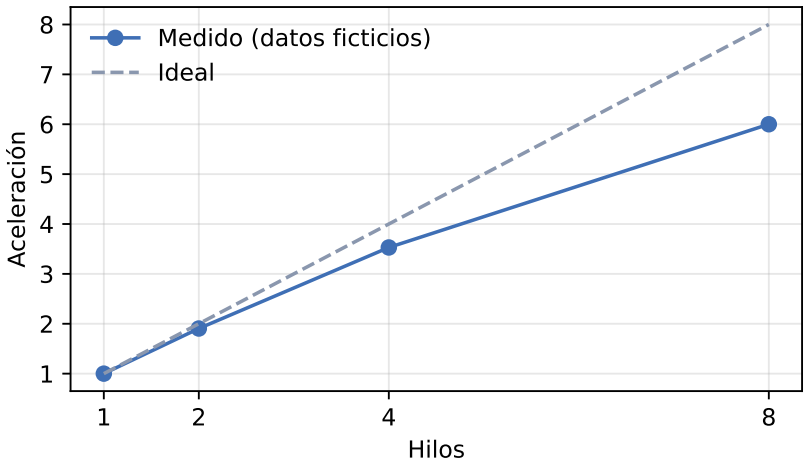


Figura 5.1: Aceleración frente al número de procesos.

Según la definición 3.1, la eficiencia baja de 1,00 a 0,75 al pasar de 1 a 8 procesos, lo que es coherente con el modelo (3.1).

> **Aviso**
>
> Estos datos son inventados: sirven solo para practicar tablas y figuras.

6 Conclusiones

6.1 Conclusiones

Se ha presentado un diseño modular y se ha evaluado con datos ilustrativos. Los resultados muestran una aceleración creciente aunque con eficiencia decreciente.

6.2 Trabajo futuro

Quedan como trabajo futuro probar con conjuntos de datos reales y comparar con otros sistemas.

A Instalación

```
$ python -m venv venv
$ source venv/bin/activate
$ pip install -r requirements.txt
```

B Conjuntos de datos

Nombre	Tripletas	Tamaño (MB)
pequeno	10 000	1
mediano	1 000 000	80

Cuadro B.1: Conjuntos de datos de prueba (ficticios).

Bibliografía

[1] A. González-Escribano y D. R. Llanos, "Trasgo: a nested-parallel programming system", *The Journal of Supercomputing*, vol. 58, n.º 2, págs. 226-234, 2011. DOI: 10.1007/s11227-009-0367-5.

[2] Grupo de investigación Trasgo, Universidad de Valladolid. "Sitio web del grupo Trasgo". (), dirección: https://trasgo.infor.uva.es/ (visitado 26-10-2026).

[3] J. D. Fernández, M. A. Martínez-Prieto, C. Gutiérrez, A. Polleres y M. Arias, "Binary RDF representation for publication and exchange (HDT)", *Journal of Web Semantics*, vol. 19, págs. 22-41, 2013. DOI: 10.1016/j.websem.2013.01.002.

[4] D. L. Parnas, "On the criteria to be used in decomposing systems into modules", *Communications of the ACM*, vol. 15, n.º 12, págs. 1053-1058, 1972. DOI: 10.1145/361598.361623.

[5] L. Lamport, *LaTeX: A Document Preparation System*, 2.ª ed. Addison-Wesley, 1994.